Informe de Firmware y Aplicación - Proyecto Semáforo Hackathon

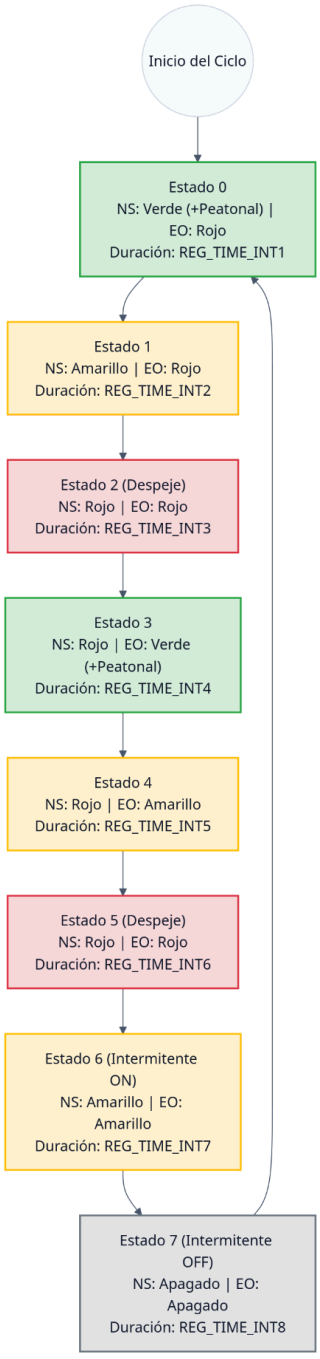
1. Arquitectura del Firmware

El firmware de la central electrónica fue desarrollado sobre la plataforma STM32 Blue Pill utilizando el sistema operativo de tiempo real FreeRTOS. El objetivo es controlar las fases de una intersección (Norte/Sur y Este/Oeste), verificar el estado del sistema y ofrecer capacidades de operación como Esclavo Modbus.

Diagrama de Flujo del Procedimiento

El flujo general de ejecución del firmware sigue estos pasos principales:

1. Inicio del Sistema: Se actualiza el reloj del sistema en la función main.
2. Inicialización de Hardware: Se configuran los pines GPIO (salidas para luces y entradas analógicas), el conversor ADC (para el sensor LDR y lectura de corriente) y el periférico USART1 mediante la función vHardwareInit.
3. Creación de Recursos FreeRTOS: Reserva de memoria para la cola de recepción UART (xUartRxQueue).
4. Instanciación de Tareas: Se crean tres tareas concurrentes con distintas prioridades: vStateMachineTask, vReadInputsTask y vSerialCommTask.
5. Ejecución (Scheduler): El sistema cede el control al planificador de tareas, el cual administra la ejecución según prioridades, eventos y tiempos.



Descripción de Tareas y Funciones Asociadas

- vStateMachineTask:
    - Descripción: Tarea principal que ejecuta la máquina de estados finitos para controlar la secuencia de los semáforos. Gestiona una secuencia normal de 8 intervalos temporales configurables e

implementa lógicas de excepción como el modo Todo Rojo forzado o el modo Destellante, que se activa de forma manual o automáticamente por la oscuridad detectada por LDR.
    - Funciones llamadas: Llama a vHardwareSetLights para activar físicamente las salidas hacia los LEDs de acuerdo a la fase, y hace uso de vTaskDelay para sostener los intervalos de tiempo y muestrear cambios rápidos.
- vReadInputsTask:
    - Descripción: Monitorea de manera periódica (cada 100 ms) los periféricos y condiciones de falla. Evalúa los niveles de luz mediante el LDR para habilitar el modo nocturno, simula o estima la carga de la batería e infiere la corriente consumida para detectar un estado de sobrecorriente o de lámpara quemada.
    - Funciones llamadas: usAdcReadChannel para leer conversiones analógico-digitales mediante sondeo por software, y xTaskGetTickCount en conjunto con vTaskDelayUntil para mantener la constancia en el ciclo de muestreo de la tarea.
- vSerialCommTask:
    - Descripción: Procesa la entrada de comunicación asíncrona a través del protocolo Modbus RTU. Expone el vector global de variables g_modbus_registers permitiendo que un equipo externo pueda realizar operaciones de lectura y modificación.
    - Funciones llamadas: xQueueReceive para obtener datos alimentados asíncronamente por la interrupción UART USART1_IRQHandler, usModbusCRC para la comprobación de integridad y vModbusProcessFrame para decodificar los comandos maestro (funciones 0x03 y 0x06).

2. Aplicación de Control (Master)

Para la validación del ecosistema se diseñó una herramienta de software móvil asistida por Inteligencia Artificial.

- Propósito y Funcionamiento: El propósito de esta aplicación es actuar como dispositivo Maestro (Master) en la red Modbus. Dentro del entorno de pruebas específico del hackathon, sirve puntualmente para corroborar y auditar la funcionalidad de un único esclavo, transmitiendo configuraciones en caliente y permitiendo visualizar los registros y banderas de estado.

- Interfaz de Usuario:

